DA3408 Assignment 3 — Spark vs. Ray: The Data Engineering Duel

Aakash Aadhithya (DA24B028)

Code, logs and screenshots: `https://github.com/AakashAadhithya/da3408-a3`

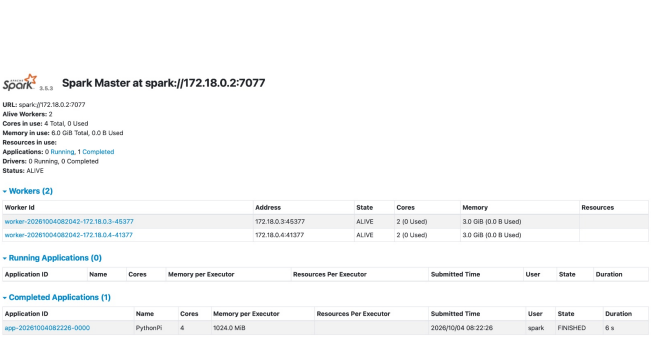
Summary

An identical cleansing pipeline (ingest → normalise → nulls → validity → exact dedupe → Python UDF → two zone joins → hourly aggregate → Parquet) was run on 36 months of NYC Yellow Taxi data (1.9 GB, 2022–2024) on a 2-worker Spark cluster and a 2-worker Ray cluster with identical resources. Both produce the *same* 106,839,585 output rows (verified by an order-independent hash). **Spark was faster** (median 604.7 s vs. 659.6 s), kept the cores ∼95% busy and needed no tuning; **Ray** ran the Python UDF at less than half Spark's per-row cost (no JVM↔Python boundary) but needed six memory-related redesigns to finish at all on 4 GB workers, and its resulting month-by-month execution used only ∼46% of the CPUs.

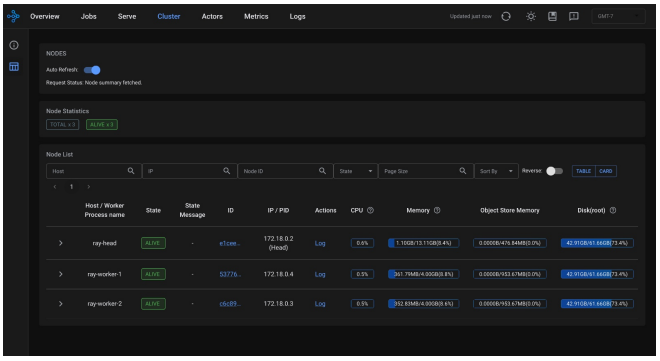
1 Infrastructure and Cluster Orchestration

Host: MacBook Air M5 (10 cores, 24 GB) → UTM Ubuntu ARM64 VM (6 vCPU, 14 GB RAM). Each framework runs as three Docker Compose containers on one bridge network; only one cluster runs at a time.

	Spark 3.5.3 (standalone, Python 3.8.10)	**Ray 2.59.0** (Ray Data, Python 3.11.16)
Coordinator	1 master + driver (`--driver-memory 2g`), no limit	1 head, `--num-cpus=0` (runs no tasks), no limit
Workers	2 × (2 cores, container 4 GiB, `SPARK_WORKER_MEMORY=3g`)	2 × (2 CPUs, container 4 GiB, object store 1 GB)
Per-task memory	executor heap 2500 MB ⇒ (2500−300)×0.6 = 1320 MiB unified pool	Python worker processes + 1 GB shared-memory object store
Data	`/data` bind-mounted into all containers	same



(a) Spark Master UI: 2 ALIVE workers.



(b) Ray Dashboard: head (0 CPU) + 2 workers.

Figure 1: Cluster evidence (more in `screenshots/`: executors on both workers, smoke-test jobs, resource views).

2 Data and Pipeline Parity

Schema drift. Reading footers showed two physical schemas: 2022-01…2023-01 (13 files) vs. 2023-02…2024-12 (23 files) differ in `VendorID`/`PULocationID`/`DOLocationID` (int64 vs int32), `passenger_count`/`RatecodeID` (double vs int64), `store_and_fwd_flag` (string vs large_string) and `airport_fee` vs `Airport_fee`. Both pipelines group files by footer schema, read each group, rename/cast to one frozen target schema and union — inside the timed run.

Cleansing contract (identical in both): drop rows with any null (the nulls come as a block of 5 columns); keep distance, fare, total > 0, $1 \le$ passengers ≤ 6, $1 \le$ RatecodeID ≤ 6, 60 s $\le$ duration $\le$ 3 h, pickup in [2022-01-01, 2025-01-01); exact full-row dedupe; ISO day-of-week; the Python UDF `trip_features(pickup, dropoff, distance)` → (duration_min, avg_speed_mph), then speed $\le$ 100 mph; left joins to the zone table on pickup and dropoff (read with Python `csv` in both, so "N/A" is never turned into null); `speed_vs_hour` = speed / mean speed of that pickup hour. All time logic is timezone-free (`timestamp_ntz` / naive Arrow timestamps).

Verification. `compare.py` checks row count, schema (names, types, order), the wrapping sum of 64-bit row hashes over all exact columns (an order-independent multiset fingerprint, which covers the UDF outputs bit-exactly), and the two sum-order-dependent float columns with tolerance 10^{-9}.

Dev slice (2023-01 + 02)	Spark	Ray	**Full data** (36 files)
0 ingested	5,980,721	5,980,721	rows: 106,839,585 (both) — PASS
1 nulls dropped	5,832,161	5,832,161	schema (30 cols, types, order) — PASS
2 validity filters	5,572,689	5,572,689	row-multiset hash (exact columns) — PASS
3 exact dedupe	5,572,689	5,572,689	hourly mean max $\lvert\Delta\rvert = 7.1 \times 10^{-14}$ — PASS
4 UDF + speed ≤ 100	5,572,487	5,572,487	$\sum$speed_vs_hour rel. diff 2.5×10^{-15} — PASS

Table 1: Parity: identical counts at every stage, and identical full-scale outputs. Re-checked on later runs (max $\lvert\Delta\rvert$ 4.4×10^{-14}).

3 Performance Analysis

Each framework was run 3 times on the full data, alternating frameworks; the timer starts after the cluster connection and stops when the Parquet export completes. `docker stats` was sampled every ∼1 s (`monitor.sh`); all logs and CSVs are in `bench/`. Framework UIs confirm the timings (Ray job 11m 0s for run 3 vs. logged 659.6 s; Spark job durations in `screenshots/`).

	Run 1	Run 2	Run 3	**Median**	CPU (busy)	1 worker	2 workers	Driver
Spark	604.65	692.06	598.38	**604.65 s**	381%	2.73 GiB	5.32 GiB	2.71 GiB
Ray	620.94	761.04	659.64	**659.64 s**	186%	3.46 GiB	6.73 GiB	1.94 GiB

Table 2: Wall clock (s) and resources: average CPU while busy (of 400%) and peak memory (medians over 3 runs). Run 2 was ∼15% slower for *both* frameworks, pointing to VM-level noise. A 4th Spark run used only for screenshots took 556.45 s.

Reading the CPU timeline. Spark holds a ∼400% plateau that maps onto its jobs: job 38 (scan, cleanse, dedupe map side; 1.5–2.0 min), job 39 (dedupe reduce + Python UDF + checkpoint write; 5.2–5.6 min, 200 shuffle partitions coalesced to 100 by AQE) and the final join/export at ∼375%. It also shows ∼22 s at the start where the executors are idle while the driver works (schema jobs end at 11:14:43, job 38 starts 11:15:06; the same gap appears in run 4), most likely query planning of the wide plan — not verified. Ray oscillates between ∼120% and ∼380% with one burst every ∼12 s in phase B (445 s/36 months ≈ 12.4 s): each month job bursts during dedupe+UDF, then drops while the next small job shuffles and writes (Ray phases, run 3: A 74.9 s, B 445.4 s, C 139.4 s). **Approximate CPU work:** Spark $3.81 \times 604.7 \approx 2{,}300$ core-s vs. Ray $1.86 \times 659.6 \approx 1{,}230$ core-s — Ray did the same work with about half the CPU time, but left half the cluster idle.

Memory. Spark executors are bounded by the JVM heap and peak at 2.73 GiB of 4. Ray workers peak at 3.46 GiB, ∼0.35 GiB below Ray's OOM-kill threshold (95% of the container) — the margin that the tuning in Section 5 was fighting for.

4 UDF Deep-Dive: JVM overhead vs. Python-native

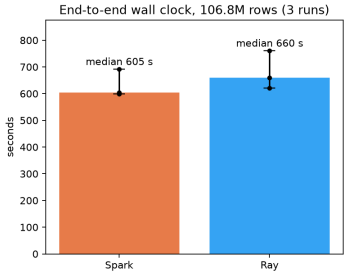
The UDF stage was isolated on 20,252,892 rows (3 columns, input cached in both), in three variants per framework, median of 3, 4 cores. All six variants return identical checksums.

	Spark	Ray	Overhead vs. native (Spark / Ray)	
Native (Spark expressions / pyarrow.compute)	1.08 s	0.54 s	—	—
Row-at-a-time Python (`trip_features`)	26.42 s	12.33 s	25.34 s	11.79 s
Vectorised (`pandas_udf` / pandas batch)	4.02 s	0.53 s	2.94 s	≈0 s

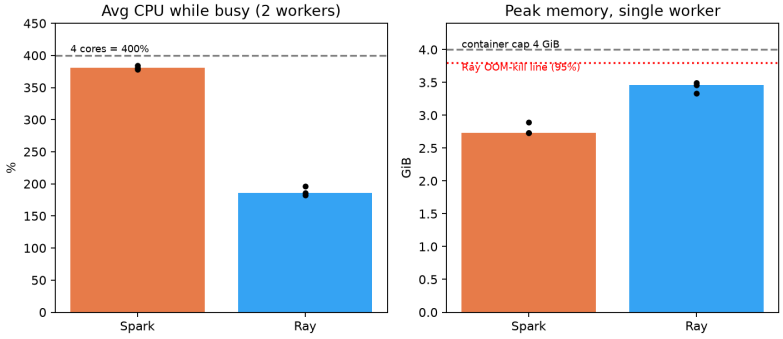
Table 3: UDF micro-benchmark (`spark/udf_bench.py`, `ray/udf_bench.py`).

Where the time goes (CPU time per row = overhead × 4 cores / rows).

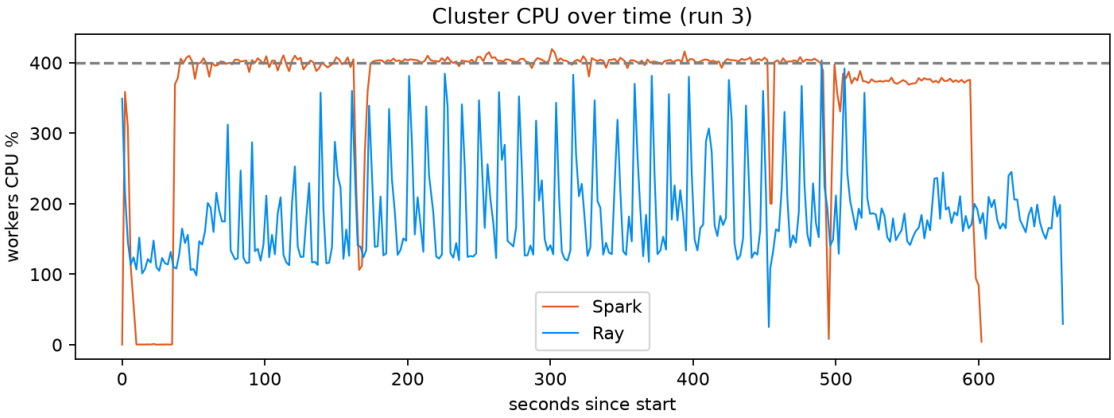
- The function itself costs ≈0.14 µs/row: run alone in plain Python (no framework) it processes 7.04 M rows/s on Python 3.8 and 7.12 M rows/s on 3.11. The interpreter-version confound is therefore ≈1%.
- **Ray**: 11.79 s ⇒ ≈2.3 µs/row, almost all of it Arrow→Python object conversion (`to_pylist`, datetimes) and back.
- **Spark**: 25.34 s ⇒ ≈5.0 µs/row: the same object conversion *plus* ≈2.7 µs/row for the JVM↔Python boundary — rows pickled in the JVM, sent over a socket to a Python worker, unpickled, results pickled and decoded back in the JVM (`BatchEvalPython`).



(a) End-to-end time.



(b) Average busy CPU and peak worker memory.



(c) Summed worker CPU over time, run 3.

Figure 2: Benchmark results.

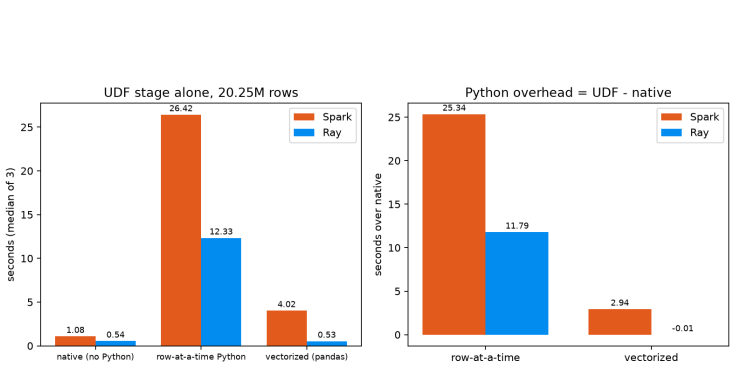
- **Vectorised**: Arrow batching cuts Spark's overhead 8.6× (25.34→2.94 s). The plan reports 23.3 MiB sent and 15.3 MiB returned per 1 M rows — exactly 24 B/row (two timestamps + one double) and 16 B/row (two doubles): raw columnar data, but it still has to leave the JVM and come back. Ray's data is already Arrow inside a Python process, so its pandas variant costs nothing over native.

Conclusion: Ray's "Python-native" advantage is real and measurable — about 2.7 μs per row for row-wise Python and the entire ∼3 s Arrow round trip for vectorised Python — and it does not come from the interpreter. In the full pipeline this is ≈530 (Spark) vs. ≈250 (Ray) CPU-seconds of UDF work, a large part of the CPU-time gap above. (Caveat: Spark's `pandas_udf` used its default 10k-row Arrow batches, Ray 250k-row batches.)

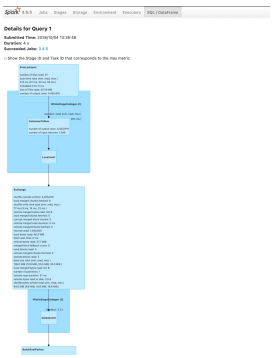
5 Scaling Ray on 4 GB Workers — Performance Tuning Note

Spark ran the pipeline with default settings apart from `--executor-memory 2500m` and `--driver-memory 2g` (the driver's default 1 GB heap caused an OOM). Ray did not complete with a direct translation. Each change below was **suggested by an LLM (Claude) after I ran the diagnostics shown**, and every change kept the output identical (parity re-verified after each):

Observed problem (evidence)	**Change**
19-column `groupby().count()` dedupe: >4 min on the dev slice, still running; 16 partitions did not help	Bucketed dedupe: shuffle by `pickup_seconds mod 64`, then exact distinct per bucket in Arrow C++ (6.7 s)
Full data: global shuffle stalled for >3 h (4431/4432 tasks), raylet crashed 18 min in, ∼20 GB spilled, avg CPU 20%	Dedupe month by month (exact, because duplicates share their pickup time and hence pickup month)
`write_parquet(partition_cols=...)` triggered OOM kills	Pure-Arrow per-block partitioned writer
Whole-block batches copied 3–4× inside fused tasks → OOM	`batch_size`=250k rows, 64 MB target blocks
Month-sized checkpoint files; a retried task could duplicate rows	Bounded files, content-hashed names, atomic rename (retry-safe)
A single read task used 2.28 GB (Ray's "top memory users" log)	One file per read task (`override_num_blocks`)



(a) UDF times and Python overhead.



(b) `BatchEvalPython`.



(c) `ArrowEvalPython`.

Figure 3: UDF results and Spark physical plans: the row UDF and the pandas UDF each add a Python node; the native query has none.

Spark needed none of this: its reader splits input into bounded partitions, its shuffle spills to disk, AQE coalesces partitions and `dropDuplicates()` is native. The month-by-month loop is also why Ray uses only ∼46% of the CPUs.

6 Winner, and AI-first vs. BI-first

Winner for this workload: Spark — 8% faster wall clock, ∼95% CPU utilisation, bounded memory, and it worked out of the box on a small, memory-constrained cluster. Its cost is the JVM↔Python boundary, which matters only where Python UDFs dominate.

BI-first project (SQL-style ETL, joins, aggregations, dedupe, reporting): choose **Spark**. Its optimiser, AQE, native relational operators and robust spilling are exactly what this pipeline needed, and the heavy steps never left the JVM.

AI-first project (Python feature code, embeddings, model inference, GPU stages): choose **Ray**. Python and Arrow are native there, so vectorised Python costs nothing extra and row-wise Python costs less than half of Spark's; tasks and actors map naturally onto ML stages. Plan for more memory per worker, or explicit block and batch sizing, for shuffle-heavy steps.

Threats to validity. One VM on one laptop (shared disk, virtualisation noise; 3 runs per framework, medians); CPU and memory come from ∼1 s `docker stats` samples; Ray's design was adapted to fit memory, so "Ray" here means Ray with a per-month dedupe; Spark workers ran Python 3.8 and Ray 3.11 (measured effect ≈1%); the UDF benchmark used a separate image (`da3408/spark-arrow`, adds pandas and pyarrow), while the full Spark runs used the stock `apache/spark:3.5.3-python3` image, whose Python UDFs do not use Arrow.

AI Usage and Attribution

Anthropic Claude was used throughout the development of this project as an AI-assisted programming and tutoring tool. I used Claude interactively to understand and translate PySpark concepts into Ray Data, generate and iterate on initial code and Docker Compose configurations, troubleshoot implementation and deployment issues, and suggest benchmarking, plotting, and Ray configuration approaches. The interaction was iterative: I provided the project requirements, reviewed the generated suggestions, integrated or modified the code, and tested the resulting implementation myself.

Claude was also used to help structure and refine parts of this report based on my own implementation and experimental findings. It was not used to generate or fabricate experimental results. All benchmarks were executed by me, and the reported figures and tables are based on my own runs, with logs and Docker statistics recorded in bench/ and results cross-checked against the Spark UI and Ray Dashboard screenshots. I made the final decisions regarding the implementation, experimental setup, tuning, interpretation of results, and contents of the report.